Ders 17 Genişletilmiş Notu: Çok İş Parçacıklı Programlama (pthread) ve void* ile Genel Fonksiyonlar

Ders 17 Genişletilmiş Notu: Çok İş Parçacıklı Programlama (pthread) ve void* ile Genel Fonksiyonlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tek Akıştan Çok Akışa
- 2. Çok İş Parçacığı Ne İşe Yarar?
- 3. Senkron ve Asenkron
- 4. Süreç ve İş Parçacığı
- 5. Bağlam Değiştirme
- 6. POSIX Thread Kütüphanesi
- 7. İlk Örnek: main ve Bir İş Parçacığı
- 8. Derleme
- 9. Aynı Fonksiyonu Çalıştıran İki İş Parçacığı
- 10. Değer Döndürme ve pthread_join
- 11. Beş İş Parçacıklı Örnek ve Argüman Gönderme
- 12. Genel Fonksiyonlara Giden Yol
- 13. Genel Swap Fonksiyonu
- 14. Dizinin İlk ve Son Elemanını Değiştirmek
- 15. Dizi Döndürme ve Genetik Algoritma Örneği
- 16. Genel Kabarcık Sıralaması
- 17. C'de Tür Bilgisi ve void* Üzerine Son Söz

1. Tek Akıştan Çok Akışa

Bir önceki derste çok dosyalı programların nasıl derlendiğini, `make` aracını ve `makefile` dosyasını konuşmuştuk. Bu derste çok iş parçacıklı (multi-thread) programlamaya geçiyoruz.

1.1 Şimdiye Kadarki Programlar: Tek Akış

Şu ana kadar yazdığımız programlar `main`'in bulunduğu yerden başlar, `return 0` satırına doğru akar ve orada biter. Araya fonksiyon çağrıları girer: `main` bir yerde kesilir, `f` çalışır, `f` bitince `main` kaldığı yerden sürer. Sonra `g`'ye, sonra `h`'ye gidilip geri dönülür. Hoca bu akışı önce fareyle ekranda çizdi. Düz metinle şöyle gösterilebilir:

```
main ---> f ---> main ---> g ---> main ---> h ---> main ---> return 0
```

Akış dallanıyor gibi görünse de herhangi bir anda yalnızca bir yer çalışır. Dolayısıyla bu yapı tek akışlıdır. Hocanın deyişiyle, herhangi bir T anında iki fonksiyonun yan yana çalıştığı bir durumu şimdiye kadar görmedik.

1.2 Görev Paralelliği

Çok iş parçacıklı yapının çıkış noktası şu sorudur: Elimizdeki işlemciyi daha verimli kullanmak için programımızı iş parçacıklarına bölsek, aynı anda birden fazla akışla çalışabilir miyiz?

Programı zaten mantıksal olarak fonksiyonlara bölüyoruz. Bir adım daha atıp "bu fonksiyon şu fonksiyonla aynı anda çalışabilir" diyebiliriz. Buna işlerin paralelleştirilmesi, yani görev paralelliği (task parallelism) denir.

Hoca farkı `f1` ve `f2` ile anlattı. Kodda önce `f1`'i, sonra `f2`'yi çağırırsak `f1` tamamlanır, ondan sonra `f2` tamamlanır. Çok iş parçacıklı yapı ise şunu sağlar: `f1`'in belli bir kısmı yapılırken `f2`'nin de belli bir kısmı yapılabilir.

1.3 Tek İşlemcide Nasıl Olur? Sanal ve Gerçek Paralellik

Akla hemen şu soru gelir: İşlemleri yapacak birim tek ise iki iş nasıl aynı anda yürür? Mantık, işlemcinin GHz seviyesinde çok hızlı çalışmasına dayanır.

- **Sanal paralellik:** İşlemci tek ise zaman paylaşımı yapılır. Birinin bir kısmı, sonra öbürünün bir kısmı çalışır. Geçişler bir saniyenin çok altında olduğu için gözle ayırt edilemez. Hoca, eskiden beri var olan çok görevli (multitask) çalışmanın da böyle olduğunu söyledi.
- **Gerçek paralellik:** İşlemci birden fazlaysa bir işlemcide `f1`'in rutinleri çalışırken öbür işlemcide `f2`'nin rutinleri çalışır.

İşin ayrıntıları ve donanımsal temelleri bu dersin konusu değildir. Süreç, program, iş parçacığı kavramları, kısıtları ve özellikleri ileride İşletim Sistemleri ve Bilgisayar Organizasyonu derslerinde yerine oturacak. Bize şimdilik gereken, bir program içinde iş parçacıklarının nasıl oluşturulup yönetileceğidir.

2. Çok İş Parçacığı Ne İşe Yarar?

Birkaç işin paralel çalışması, kullandığımız uygulamaların çalışma biçimidir. Hoca üç örnek verdi:

- **Zoom:** Hoca bir yandan yazı yazıyor, bir yandan konuşuyor, bir yandan da sunumu aktarılıyor. Tek bir uygulama var, ama altında "kol kola, el ele" görev yapan pek çok fonksiyon var. Yazıyla ilgili iş, konuşmanın aktarılması ve belki gürültü giderimi ayrı fonksiyonlardır.
- **Word:** Yazı yazarken hatalı kelimelerin altı kırmızıyla çizilir. Önce bütün yazıyı bitirip elinizi klavyeden çekmeniz, ancak ondan sonra yazım denetiminin (spell check) başlaması gerekmez. Hocaya göre Word artık anlamsal uygunluğa da bakıyor.
- **Web tarayıcısı:** Chrome, Opera ya da Edge'de aynı anda birkaç sekme açıktır. Birinde müzik dinlerken öbüründe e-postanızı okuyabilirsiniz.

Tek bir uygulamanın altındaki bu farklı işlere çok iş parçacıklı yapı diyoruz. C dilinde de bunu sağlayan olanaklar vardır.

3. Senkron ve Asenkron

İş parçacıklarıyla yürütme aslında yürütmenin (execution) asenkron olması demektir. Senkron ve asenkron kavramlarını pek çok yerde göreceğiniz için hoca önce sınıfa bunların ne anlama geldiğini sordu.

Senkron yapı. Belli şeyleri belli kontrollerle birbiriyle denkleştirmek, eşitlemek demektir. Aslında bir protokol belirlersiniz. Hocanın ilk örneği telefondaki senkronizasyon ayarıydı. E-posta hesabınızı telefonla senkronize etmezseniz gelen posta sunucuda kalır, telefondaki uygulamaya düşmez. Protokol şudur: Önce posta gelir, gelme işi biter, ondan sonra uygulamada görünür. İkinci örnek eski telsizlerdir: Bir kişi düğmeye basar, konuşur, ses karşıya gider, ondan sonra dinler.

Asenkron yapı. Örnek, bu dersin kendisidir: Hoca bir yandan konuşurken siz bir yandan dinleyebiliyorsunuz. Bunu senkron bir gönder-al (send, receive) mantığıyla yapsaydık hoca konuşur, söyledikleri bir yerde tamponlanır (buffer), konuşma bitince "artık duymaya başlayın"

denir, siz de gecikmeli olarak aynı sırayla duymaya başlardınız. Bu tür iletişimde senkron yapı pek sağlıklı değildir. Yine de kullanıldığı yerler vardır, kullanılmadığı yerler vardır.

Programlardaki asenkronluktan kasıt şudur: Programda $f_1, f_2, \ldots, f_n$ gibi n tane fonksiyon olsun. Bunlardan herhangi k tanesini seçip paralel çalışabilir diyorsunuz. Seçtikleriniz arasında mesela `f3`, `f8` ve `f20` varsa bunların belli kısımları beraber çalışacaktır. **Kimin ne kadar ve nasıl çalıştığını bilmiyoruz.** Öte yandan bu n fonksiyonun bazılarının biri önce, biri sonra çalışması gerektiğini de biliyoruz. Buna dikkat etmek gerekir.

Hoca, gerçek hayat asenkrona çok yakın olduğu ve uygulamalar da bu senaryoları canlandırdığı için çok iş parçacıklı yapılara ihtiyaç duyulduğunu söyledi.

4. Süreç ve İş Parçacığı

4.1 Süreç (Process)

Yazdığınız programı çalıştırdığınızda çalışan programa, yani aktif varlığa süreç (process) denir. Birden fazla süreç oluşturmak mümkündür. Süreçler birbiriyle haberleştirilerek büyük bir uygulama da kurulabilir.

Hoca Word örneğini buna göre değiştirdi: Ekrana yazı yazmak için bir süreç, yazım denetimi için bir başka süreç, yazıların şeklini değiştirmek için bir başka süreç olsun. Bu yol senkronizasyon ve veri haberleşmesi açısından maliyetlidir. Çünkü her sürecin kendine ait bir bellek bölgesi vardır. Üç süreç oluşturursanız üç ayrı bellek bölgesi olur. Dizilerde aldığımız bazı hataları hatırlayın: Bir süreç, başka bir sürecin bellek uzayına erişemez. Bu da süreçler arası haberleşmeyi zorlaştırır ve ek kontroller getirir.

Bu yüzden zaman içinde çok sayıda süreç yerine, bir süreç içinde birden fazla iş parçacığı oluşturma esasına geçilmiştir. Süreçler ile iş parçacıkları arasındaki diğer ayrıntıları hoca şimdilik geçti.

4.2 İş Parçacıklarının Zorlukları

İş parçacıkları her zaman her anlamda kullanışlı olmayabilir. Hoca iki sorun saydı:

- **İşletim sistemine bağımlılık:** Donanımla doğrudan uğraşan bir yapı söz konusu olduğu için iş parçacığı oluşturmak işletim sisteminden izin almayı ve onunla uyumlu çalışmayı gerektirir. Windows'ta yazdığınız bir iş parçacığı kodu Linux'ta ya da başka bir sistemde çalışmayabilir.
- **Haberleşme ve eşzamanlama:** İş parçacıklarının kendi aralarında haberleşmesi gereken durumlar olabilir. Örnek, üretici-tüketici problemidir. Belli kaynaklar üretilmeden tüketilmemelidir. 10 elemanlık bir tampon düşünün: Biri buraya üretir (produce), biri buradan tüketir (consume).

Hocanın günlük hayat karşılığı uçak ya da otobüs bileti satan sitelerdir. Firma, günlük ya da aylık olarak siteye bilet ekler (üretici). Bu ekleme sürerken müşteriler de bilet almaya devam edebilmelidir (tüketici). Üretim ve tüketim arasındaki mekanizmanın gerektiği yerde senkronize edilmesi, haberleşmenin sağlanması gerekir.

5. Bağlam Değiştirme

Bağlam değiştirme (context switching) aslında İşletim Sistemleri dersinin konusudur. Hoca, bu üçüncü sınıf dersinde konuyu daha ayrıntılı anlattığını belirtti. Burada kısa bir fikir verdi.

Elimizde tek bir CPU olsun. Görevi komutu getirmek (fetch), çözmek (decode), yürütmek (execute) ve gerekirse sonucu yazmaktır (write back). Bir de `P1` ve `P2` diye iki programımız olsun. Bu iki süreç tek CPU üzerinde eşzamanlıymış gibi çalışacaktır, ama burada gerçek bir eşzamanlılık olmaz.

Hocanın benzetmesi tek kasalı markettir. Bir kasa, iki müşteri var. Ya önce biri bitecek, sonra diğeri ya da belli zaman aralıklarında parça parça işlem yapılacak. O zaman aralığının kendisi de bölünemez. Akış şöyledir:

1. P1 işlemciye girer ve T_1 süresi kadar çalışır. Bu sırada P2'nin işlemciye girmesi mümkün değildir.
2. Arada kısa bir boşluk olur: P1 çıkar, P2 girer.
3. P2, T_2 zamanına kadar çalışır.
4. P1 kaldığı yerden devam eder, sonra P2 kaldığı yerden devam eder.
5. Bu, işleri bitip işlemciden çıkana kadar sürer.

```
CPU: [ P1 ] bosluk [ P2 ] bosluk [ P1 ] bosluk [ P2 ] ...
     ----- zaman ----->
```

Bir sürecin içeriği (bağlamı) alınıp değiştirildiği için bu işleme bağlam değiştirme (context switch), aradaki boşluğa, yani P1'in çıkıp P2'nin işlemciye girmesi için geçen süreye de bağlam değiştirme süresi (context switching time) denir.

Bu anlatım tek işlemci içindir. Yeterince CPU varsa P1 bir işlemcide çalışırken P2 başka bir işlemcide çalışabilir. Hoca bu duruma girmedi.

6. POSIX Thread Kütüphanesi

6.1 Kütüphane ve İş Parçacığı Kimliği

C'de iş parçacığı oluşturmak için, donanıma kadar inen özellikleri destekleyen `pthread` kütüphanesini kullanırız. Programa `#include <pthread.h>` satırını ekleyerek iş parçacığı oluşturma, sonlandırma ve iş parçacığı fonksiyonu yazma gibi işleri yapabiliriz.

İşlemler iş parçacığının kimliği (thread ID) üzerinden yapılır. Bir dizinin ya da dosyanın kendine ait işaretçisi olması ve işlemleri onun üzerinden yapmanız gibi, her iş parçacığının da kendine özgü bir kimlik değeri vardır.

> **Ders sonrası not:** Hoca kimliğin genellikle pozitif bir tam sayı olduğunu, tam sayı olmayan durumların şimdilik dikkate alınmayabileceğini söyledi. Bu makinede `sizeof(pthread_t)` 8 çıktı (gcc ile denendi). Büyük olasılıkla POSIX `pthread_t`'nin iç yapısını tanımlamaz, yani tam sayı olması garanti değildir. Bu yüzden iki kimliği `==` ile değil, Bölüm 9'daki `pthread_equal` ile karşılaştırın.

6.2 Fonksiyon Adlarındaki Alt Çizgi

Hoca slayttaki kırmızı yazılı fonksiyon adlarını gösterip dikkat çeken ortak noktayı sordu. Cevap alt çizgiydi: `pthread_create`, `pthread_join`, `pthread_exit`. Kelimeleri alt çizgiyle ayırmak POSIX standardından gelir. Hoca bunu, IEEE'nin 754 kayan nokta gösterimi standardına benzetti. Deve hörgücü (camel case) gibi başka yazım biçimleri de vardır, ama burada kullanılan POSIX biçimidir. Kütüphanenin "POSIX thread" diye de geçmesi bundandır.

6.3 pthread_create

Fonksiyonun bildirimi şöyledir:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

Dört parametresi vardır:

- **`thread`:** `pthread_t` türünden bir işaretçi. Kimlik buraya yazılır. Elimizde `pthread_t id;` diye bir değer varsa `&id` veririz. Çünkü `id`'nin değeri değişecektir: İşletim sistemi bir iş parçacığı oluşturup ona kimlik numarası atayacak ve bu numarayı oraya yazacak. `int i` ile `int *p` ayrımında olduğu gibi, biri değer tutar, öbürü adres tutar. Daha sonra iş parçacığını öldürmek ya da başka bir işlem yapmak gerekirse bu kimlik kullanılır.
- **`attr`:** İş parçacığının özelliklerini gösteren işaretçi. Varsayılan özellikler için `NULL` yazılır.
- **`start_routine`:** İş parçacığının çalıştıracağı fonksiyonun işaretçisi. Fonksiyon işaretçilerinin nasıl tanımlandığını daha önce görmüştük. Buraya fonksiyonun adı verilir.
- **`arg`:** Bu fonksiyona gidecek argüman. `void*` türündedir. Argüman yoksa `NULL` yazılır.

Dönüş değeri başarıda 0'dır. İş parçacığı oluşturulamadıysa (bir dosyanın açılamaması gibi bu da olabilir) bir hata numarası döner.

Hoca bu kuralların “çok sert” olduğunu vurguladı. Bir iş parçacığı fonksiyonu tam olarak şu kalıpta olmalıdır:

```
void *fonksiyon(void *arg);
```

Neden böyle? Çünkü bu fonksiyon genel (generic) çalışacaktır. İstediğiniz türde parametre alabilir, istediğiniz türde değer döndürebilirsiniz. Gelenler ve gidenler her türden olabilsin diye `void*` kullanılır. Bu nedenle bir tam sayıyı doğrudan veremezsiniz, tür dönüşümü (cast) yaparak verirsiniz.

İş parçacığı özellikleri bu dersin konusu değildir. Hocaya göre bu, İşletim Sistemleri dersinin konusudur ve yalnız özelliklerin düzenlenmesi bile yarım dönemlik bir iş olabilir. Biz genellikle varsayılan değerleri kullanıyoruz, ama varsayılanlar her zaman işe yaramayabilir.

7. İlk Örnek: main ve Bir İş Parçacığı

İlk örnekte bir iş parçacığı oluşturuluyor. Dönüş değeri 0 değilse hata yazılıp `main`’den çıkılıyor. 0 ise `main` sonsuz bir `while` döngüsüne giriyor. İş parçacığı fonksiyonunun içinde de sonsuz bir döngü var.

```
#include <stdio.h>
#include <pthread.h>

void *threadFunction(void *arg) {
    while (1) {
        printf("I am thread function\n");
    }
    return NULL;
}

int main(void) {
    pthread_t id;
    int ret = pthread_create(&id, NULL, threadFunction, NULL);

    if (ret != 0) {
        printf("Thread olusturulamadi\n");
        return 1;
    }
    while (1) {
        printf("I am main function\n");
    }
    return 0;
}
```

Hoca ekranda ne görüleceğini sordu.

Soru: Hangisi önce yazar, sıra öngörülebilir mi? Bir öğrencinin cevabı, ikisinin gidip geleceği ve karışık olabileceğiydi. **Cevap:** Karışık olması gerekir ve sıra hakkında bir fikrimizin olmaması da gerekir. Programın artık iki akışı vardır. `main` kendi içinde `while` döngüsünden dolayı sonsuza kadar akar, iş parçacığı fonksiyonu da kendi içinde akar. İkisi kol kola gider. Bunları işlemciye sokup çıkaran işletim sistemidir ve bunu kendi ölçütüne göre yapar. Süreç birdir, ama altında iki yürütme akışı vardır.

Hoca kodu derste çalıştırdı. Satırlar çok hızlı aktığı için çıktıyı izlemek zordu. Yazdırmadan sonra biraz uyutmak için `sleep` ekleyip akışı yavaşlattı, ama değeri küçültünce çıktı yine karmaşık göründü. Uyuyan iş parçacıklarının uyanıp işlemci sırasına yeniden gelmesi de bir süre alır.

İlk satıra bakmak öğreticidir. İş parçacığı oluşturulur oluşturulmaz “I am thread function” önce yazılır diye düşünülebilir, ama ikisi de önce yazılabilir. Oluşturma başarısız olduysa program zaten doğrudan çıkar.

> **Hocanın vurgusu:** İş parçacıklarının çalışma sırası öngörülemez. Öngörebileceğimiz bir şey yoktur.

Ek bilgi (derste anlatılmadı): Hoca `sleep` değerini milisaniye diye yazdı ve 100 gibi bir değer denedi. Bu makinede (Linux) `unistd.h` içindeki `sleep(2)` çağrısı 2 saniye bekledi (gcc ile denendi). Yani Linux'ta `sleep` saniye alır. Milisaniye alan biçim büyük olasılıkla Windows'taki `Sleep` fonksiyonudur. Kodu başka sistemde denerken birimi kontrol edin.

8. Derleme

Dev-C++ kullanıyorsanız ek bir şey yapmanız gerekmez, hocaya göre o bunu kendisi halleder. Komut satırında gcc ile derlerken ise `-lpthread` eklenir. Bu seçenek "`pthread` kütüphanesini benim uygulamama bağla (link)" demektir. Yazılmazsa program çalışmaz.

```
gcc myprogram.c -o myprog -lpthread
```

9. Aynı Fonksiyonu Çalıştıran İki İş Parçacığı

İkinci örnekte iki kez `pthread_create` yapılıyor, bu yüzden iki kimlik gerekiyor. `while` döngüsü 0'dan 2'ye kadar döner ve önce bir, sonra bir iş parçacığı daha oluşturur. Bunlara `main` de eklenince aslında kol kola yürüyen **üç** iş parçacığı vardır.

Soru: Bir öğrencinin bunları iki ayrı terminal gibi düşündüğü anlaşılıyor. **Cevap:** İki farklı terminal değil. İki terminal olsaydı o süreç olurdu. İş parçacığının ürettiğini terminale göndermesi yalnız bir çıktıdır, asıl çalışması işlemci üzerindedir. İki ayrı terminale yazdırmak ayrı bir meseledir.

İki iş parçacığı da `doSomeThing` fonksiyonunu çalıştırır. Hangisinin hangisi olduğunu anlamak için iki yardımcı vardır:

- **`pthread_self()`:** İş parçacığının kendi kimliğini verir.
- **`pthread_equal(a, b)`:** İki kimliği karşılaştırır.

Fonksiyon, kimliği `tid[0]` ise birinci iş parçacığının, değilse ikinci iş parçacığının mesajını yazar. İçindeki boş `for` döngüsü durup dururken küçük bir bekleme oluşturur. Hoca burada da `sleep` kullanılabileceğini, ayrıntısına bakılabileceğini söyledi. Çıktıda "iş parçacığı başarıyla oluşturuldu" ve "birinci, ikinci iş parçacığı çalışıyor" türünden satırlar görülür.

Her iş parçacığının aynı fonksiyonu çalıştırması zorunlu değildir. Birinci iş parçacığı `f1`, ikincisi `f2` ile ilişkilendirilebilir. Yalnız ikisi de `void*` döndürmeli ve aynı biçimde parametre almalıdır.

10. Değer Döndürme ve pthread_join

10.1 pthread_exit ile Değer Döndürmek

Şimdiye kadar iş parçacığına argüman göndermedik, ondan değer de almadık. İş bittikten sonra geriye değer döndürmek için iş parçacığı `pthread_exit` ile kapatılır ve kapanırken değişkenin adresi verilir: "kapat ve kapanırken bu değeri döndür." Örnekte değişken tam sayıdır, ama bir yapı değişkeni de olabilir. İkinci iş parçacığı da aynısını kendi değişkeniyle yapar.

10.2 main Önce Biterse

Bu programda iki iş parçacığı ve `main` olmak üzere üç akış var. Hoca sordu: `main` önce biterse ne olur? Program kapanır. Yanda çalışan iki iş parçacığı işini tamamlayamadan kalabilir. Word gibi bir uygulamanın ana programı pat diye kapanırsa diğer işlerin yarıda kalması gibi, bu iyi bir şey değildir.

Bu yüzden süreç tamamlanmadan önce iş parçacıklarının işini bitirmesine izin verilmeli, yani `main`'in bulunduğu yere "katılmaları" (join) gerekir. Akış şöyledir:

1. `main` ve iş parçacıkları birlikte çalışır.
2. `main`, katılma noktasına geldiğinde durur ve bekler.
3. İş parçacıkları işlerini tamamlar.
4. `main` biraz daha devam eder ve program tamamlanır.

10.3 Çağrı ve Tam Örnek

`pthread_join(tid[0], (void**)&ptr[0]);` şöyle okunur: `tid[0]` kimlikli iş parçacığının buraya gelmesini bekle. O gelirken `void*` olarak bir değer getirecek, bu değeri `ptr[0]` içinde tut. Hoca, `tid`'nin burada global tanımlandığını, bunun pek adetimiz olmadığını, ama iş parçacıklarıyla yapılabildiğini söyledi. Aynısı `tid[1]` için de yapılır, sonunda değerler ekrana basılır: Birinden 100, öbüründen 200 döner.

```c
#include <stdio.h>
#include <string.h>
#include <pthread.h>

pthread_t tid[2];
int ret1, ret2;

void *doSomeThing(void *arg) {
    unsigned long i;
    pthread_t id = pthread_self();

    for (i = 0; i < 100000000UL; i++)
        ;                                   /* bos dongu: kisa bekleme */

    if (pthread_equal(id, tid[0])) {
        printf("Birinci thread calisti\n");
        ret1 = 100;
        pthread_exit(&ret1);
    } else {
        printf("Ikinci thread calisti\n");
        ret2 = 200;
        pthread_exit(&ret2);
    }
    return NULL;
}

int main(void) {
    int i = 0, err;
    int *ptr[2];

    while (i < 2) {
        err = pthread_create(&tid[i], NULL, doSomeThing, NULL);
        if (err != 0)
            printf("Thread olusturulamadi: %s\n", strerror(err));
        else
            printf("Thread basariyla olusturuldu\n");
        i++;
    }

    pthread_join(tid[0], (void **)&ptr[0]);
    pthread_join(tid[1], (void **)&ptr[1]);

    printf("Birinci thread'in dondurdugu deger: %d\n", *ptr[0]);
    printf("Ikinci thread'in dondurdugu deger: %d\n", *ptr[1]);
    return 0;
}
```

Ders sonrası not: Hoca, dönen değerin iş parçacığının kendine ait bellek bölgesinden geldiğini söyledi. Örnekte `ret1` ve `ret2` global değişkendir, yani bütün iş parçacıklarının ortak kullandığı bir yerdedir. Bu yüzden iş parçacığı bittikten sonra da `main` bu değerleri okuyabilir. Kod gcc ile derlenip çalıştırıldı, 100 ve 200 değerleri alındı.

Ders sonrası not: Örnekteki `pthread_equal(id, tid[0])` karşılaştırması, `main`'in

`tid` dizisine yazmış olmasına dayanır. Büyük olasılıkla `pthread_create` kimliği yazmadan yeni iş parçacığının çalışmaya başlamayacağı garanti değildir. Boş döngü bu riski küçültür ama ortadan kaldırmaz. Daha sağlam yol, sıra numarasını Bölüm 11'deki gibi argümanla göndermektir.

Ek bilgi (derste anlatılmadı): Örnekte adresi döndürülen değişken global olduğu için sorun çıkmaz. Büyük olasılıkla iş parçacığı kendi yerel değişkeninin adresini döndürmemelidir, çünkü iş parçacığı bitince o değişken yok olur. Böyle bir durumda global değişken ya da `malloc` ile ayrılmış bellek kullanılır.

11. Beş İş Parçacıklı Örnek ve Argüman Gönderme

Hoca, paylaşacağı bir kaynaktaki farklı bir sürümü kopyalayıp çalıştırdı. Beş iş parçacığı için yer açılıyor ve bir `for` döngüsüyle sırayla oluşturuluyor. Varsayılan özellikler (`NULL`), çalışacak fonksiyon (`PrintHello`) ve argüman olarak `void*`'a dönüştürülmüş sıra numarası veriliyor: Birine 0, birine 1 gidiyor. Dönüş değeri `rc` hata gösteriyorsa bu yazılıyor.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5

void *PrintHello(void *threadid) {
    long tid = (long)threadid;
    printf("Hello world! Thread %ld\n", tid);
    pthread_exit(NULL);
}

int main(void) {
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;

    for (t = 0; t < NUM_THREADS; t++) {
        printf("Thread %ld olusturuluyor\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc) {
            printf("Hata: pthread_create %d dondurdu\n", rc);
            exit(-1);
        }
    }
    pthread_exit(NULL);
}
```

Fonksiyona `void*` olarak gelen değeri kullanmak için, `void` işaretçileri konusunda gördüğümüz gibi önce tür dönüşümü yapılır. Hoca burada `long` ya da `int` türüne dönüştürülebileceğini söyledi.

Ders sonrası not: `int t = (int)threadid;` yazımı denendi: `gcc -Wall` bu makinede (işaretçi 8 bayt, `int` 4 bayt) "cast from pointer to integer of different size" uyarısı verdi. `long` ile uyarı çıkmadı. Bu yüzden örnekte `long` kullanın.

0'dan 4'e kadar beş mesaj göreceğiz, ama hepsinin 0, 1, 2, 3, 4 sırasıyla yazılması zorunlu değildir. Hoca derste iki kez çalıştırdı, ikisinde de sıra 0, 1, 2, 3, 4 çıktı. Bunun yalnızca denk geldiğini, iş parçacıklarının oluşturulma sırasıyla hızlıca kuyruğa girdiğini söyledi. Defalarca çalıştırırsanız mesela önce 2, sonra 0, sonra 1 de görebilirsiniz. Engel yoktur.

Ders sonrası not: Örnek gcc ile 500 kez çalıştırıldı. Hello mesajları en az 8 farklı sırayla geldi: 0, 1, 2, 3, 4 sırası 149 kez, 0, 1, 2, 4, 3 sırası 307 kez görüldü. Hocanın söylediği gibi sıra sabit değildir.

> **Ders sonrası not:** Hoca `pthread_exit(NULL)` için önce "programı kapatıyor" dedi, sonra bunun `main` iş parçacığının döndüreceği değer olduğunu söyledi. Doğrusu: Bu çağrı yalnız `main` iş parçacığını bitirir, diğerleri çalışmaya devam eder. gcc ile denendi: `main pthread_exit(NULL)` ile bitince 1 saniye uyuyan iş parçacığı mesajını yazdı. `main return 0` ile bitince mesaj hiç görünmedi, süreç kapandı.

11.1 Aynı Örneğin join'li Hâli

Değer alınacaksa ya da iş parçacıklarının tamamlanması isteniyorsa `join` kullanılır. Hoca aynı örneğe yalnız `join` kısmını ekledi. Bu örnekte iş parçacığı çıkarken bir değer döndürmediği için ikinci parametre `NULL` verilebilir:

```
for (t = 0; t < NUM_THREADS; t++) {
    pthread_join(threads[t], NULL);
}
```

Önce 0. iş parçacığı, sonra 1., sonra 2., 3. ve 4. katılır, `main` en son işini bitirir. Bunu yazmazsak `main` önceden kapanıp gidebilir ve istemediğimiz sonuçlar görebiliriz.

Soru: Bu, mesajların sırayla gelmesine sebep olur mu? **Cevap:** İçeriklerin sırayla yazılmasıyla ilgisi yoktur. Çalışma sırasında 0. iş parçacığının iki satırı, 1.'nin üç satırı, öbürünün bir satırı çalışıyor olabilir ve `join` buna müdahale edemez.

> **Ders sonrası not:** Hoca aynı cevapta `join`'in iş parçacıklarının "sırayla tamamlanmasını" kesin olarak sağladığını da söyledi. Doğrusu: `join` yalnız `main`'in hangi sırayla beklediğini belirler, iş parçacıklarının bitiş sırasını belirlemez. gcc ile denendi: 0. iş parçacığı 2 saniye uyurken 1. iş parçacığı önce bitti, `main` ise yine önce 0.'nın, sonra 1.'nin `join`'inden döndü.

Hoca iş parçacıkları konusunun bu ders için kabaca bu kadarla yeterli olduğunu, daha ayrıntılısının gelecek dönemlerde görüleceğini söyleyerek ara verdi.

12. Genel Fonksiyonlara Giden Yol

Aradan sonra hoca, her dönem anlatılmayan ama vakit kaldığı için değindiği bir konuya geçti: `void` işaretçileri ve genel (generic) fonksiyonlar. Anlatım Stanford'un CS107 dersinin slaytları üzerinden yürüdü. Slaytlar adım adım ilerlediği için hoca bazı ara slaytları atladı.

Soru şudur: Mevcut bilgilerimizle, herhangi bir tipteki veriyi işleyebilen bir fonksiyon yazabilir miyiz? Örneğin n elemanlı bir dizi `int` dizisi de olabilir, başka bir dizi de. Fonksiyon hepsini desteklesin istiyoruz.

Başlangıç noktası iki `int` değeri değiştiren `swap` fonksiyonudur. İki `float`, iki karakter ya da iki yapı değişkeni değiştirmek istersek ne yapacağız? `swapInt`, `swapShort`, `swapFloat`, `swapPerson` gibi ayrı ayrı fonksiyonlar yazabiliriz. Kodu kopyalayıp `int`'leri `short` yapmak mantıklı değildir. Bir yerden sonra bu yönetilemez olur ve kafa karıştırır.

İpucunu `swapString` verir. Dizgi karakterlerden oluşur, karakterler de tam sayılara dönüştürülebilir. Öyleyse her türden veriyi bir **bayt dizisi** gibi ele alıp genel bir `swap`, genel bir yazdırma fonksiyonu yazabilir miyiz?

13. Genel Swap Fonksiyonu

13.1 Hepsinde Ortak Olanlar

Hangi tür olursa olsun, `swap` işleminde üç şey ortaktır:

- Değişkenlerin adreslerini almak gerekir, işaretçi olmadan yer değiştirme olmaz.
- Geçici (temporary) bir alan gerekir.
- Üç adımda kopyalama yapılır: birinciyi geçiciye, ikinciyi birinciye, geçiciyi ikinciye.

Hoca, sıralama örneğindeki gibi fonksiyon işaretçisinin burada tam olarak işe yaramayacağını söyledi. Orada artan ve azalan sıralama için iki yan fonksiyon, bir de üstte genel bir sarmalayıcı (wrapper) yazıyorduk. Burada ise `swap` işlemi şu üç satırın kendisidir ve her türde boyut

farklıdır: `int` 4 bayt, `short` 2 bayt. Bu yüzden farklı bir geçici alan gerekir, kopyalama da başka türlü ifade edilir.

> **Ders sonrası not:** Hoca bu sırada "karakter olsa 8 bayt" dedi. gcc ile denendi: `sizeof(char)` 1, `sizeof(short)` 2, `sizeof(int)` 4 çıktı. Bu makinede 8 bayt olan işaretçilerdir: `sizeof(void*)` ve `sizeof(char*)` 8 çıktı. Bölüm 14'te hoca da karakteri 1 birim olarak kullanıyor.

13.2 Adım Adım Kurulum

1. **Herhangi bir türde işaretçi almak için `void*`.** `int` de gelse, dizgi de gelse, `float` da gelse kabul edilir.
2. **Boyutu öğrenmek.** Geçici alan tanımlamak ya da dönüşüm yapmak için gelen verinin ne olduğunu bilmemiz gerekir, ama bilemeyiz. Çözüm, olayı türden koparıp "bu veri kaç baytlık?" sorusuna indirmektir. Bunun için üçüncü bir parametre eklenir: `size_t nbytes`.
3. **Geçici alanı bayt dizisi yapmak.** Bir değişken tanımlamak yerine gelen verinin her baytını bir karakter dizisinde tutarız.
4. **Atama yerine `memcpy`.** `void*` dereferans edilemediği için `*data1ptr` gibi bir atama olmaz. Bunun yerine `memcpy` (ya da `memmove`) kullanılır. Her atamanın yerine bir `memcpy` gelir: `temp` buraya bir tampon olur.

```
void swap(void *data1ptr, void *data2ptr, size_t nbytes) {
    char temp[nbytes];
    memcpy(temp, data1ptr, nbytes);
    memcpy(data1ptr, data2ptr, nbytes);
    memcpy(data2ptr, temp, nbytes);
}
```

Çağırırken iki adres ve boyut verilir. Değişkenin bir yapı ya da başka bir şey olmasından bağımsız olarak fonksiyon genel çalışır:

```
swap(&x, &y, sizeof(x));
```

Slaytlarda aynı fonksiyonun `int`, `short`, karakter ve yapı için çalışması gösteriliyor. Hoca, `temp` dizisinin normalde böyle yazılmayacağını, dinamik bellek ayırmayla (`malloc` ile `nbytes` kadar `char` yer açarak) yapılacağını da ekledi. Özet şudur: Veriler ister kayan noktalı, ister karakter olsun sonuçta bitlerden, bitler bir araya gelince de baytlardan oluşur. Bir karakter bir bayt olduğuna göre, kaç karakterlik yer gerektiğini söyleyip o kadar yer açar ve değişikliği bu alan üzerinden yaparız.

> **Ek bilgi (derste anlatılmadı):** Hoca `memmove`'un örtüşmelerle ilgili olduğunu söyleyip geçti. Büyük olasılıkla fark şudur: Kaynak ve hedef bölgeler çakışırsa `memmove` doğru sonucu verir, `memcpy` için bu garanti değildir. `swap` içindeki kopyalamaların biri `temp` ile yapılır, biri de iki ayrı değişken arasında olduğu için normal kullanımda çakışma olmaz.

> **Ders sonrası not:** Hoca `sizeof` için "fonksiyon" dedi. `sizeof` bir işleçtir: gcc ile denendi, `sizeof x` parantezsiz de derlendi. Değeri derleme anında türden hesaplanır, bu yüzden Bölüm 14'teki `void*` durumunda dizinin boyutunu veremez.

13.3 Soru ve Cevaplar

Soru: `size_t` veri tipi tam olarak nedir? **Cevap:** Tam sayı türünde bir değer tutar, bir şeylerin boyutunu ya da adedini saklar. Genellikle pozitif değerler tutar. Hepsini tam sayı gibi düşünebilirsiniz.

> **Ders sonrası not:** Hoca, `_t` ile biten türlerin hepsinin tam sayı olduğunu ve `_t` ekinin genellikle pozitifliği gösterdiğini söyledi. gcc ile denendi: `size_t` değerine -1 atanınca 18446744073709551615 oldu, yani işaretsizdir. Aynı eki taşıyan `ssize_t` ise -1 değerini tuttu, yani `_t` eki pozitiflik göstermez. Büyük olasılıkla bu ek "type" kısaltmasıdır. `pthread_t` için de Bölüm 6.1'e bakın.

Soru: Bu şekilde iki farklı veri tipi, örneğin bir `int` ile bir `short` yer değiştirilebilir mi? **Cevap:** Burada amaç bu değil. Amaç, her tür için ayrı ayrı `swap` yazmamaktır. `int` ile `int`, `float`

ile `float` yer değiştirir. Farklı türdeki verileri değiştirmeye gerek olabileceğini sanmıyorum, kullanıldığını da görmedim.

13.4 void* Güçlü ama Tehlikeli

Hoca, `void*`'ın güçlü ama tehlikeli olduğunu, tür denetimi (type checking) yapılmadığını slayt üzerinden vurguladı.

> **Ders sonrası not:** Hoca slayttan, C'nin bir `int`'in yarısının yer değiştirilmesine asla izin vermeyeceğini okudu. Doğrusu bunun tersidir. gcc ile denendi: Yukarıdaki `swap` iki `int` için `nbytes` değeri 2 verilerek çağrıldığında derleyici uyarı vermedi, yalnız düşük iki bayt yer değiştirdi. Tür denetimi olmadığı için C buna engel olmaz.

14. Dizinin İlk ve Son Elemanını Değiştirmek

14.1 Soru ve Tür Bağımlı Çözüm

İkinci problem: Bir dizinin ilk ve son elemanını değiştiren bir fonksiyon yazın. `int` dizisi için çözüm kolaydır. Başlangıç işaretçisini ve eleman sayısını göndeririz. İşaretçi olarak geldiği için içerideki değişiklik asıl diziyi değiştirir, 1 ile 5'in yer değiştirdiğini görürüz. Bunu az önceki `swap` ile yazarken dizinin ilk elemanını, son elemanını ve eleman boyutunu göndeririz.

Ama bu `swap_ends_int` yalnız `int` için çalışır. Karakter ya da başka dizi için de ayrı fonksiyon gerekir. `swap`'ten kurtulduk ama başladığımız noktaya geri geldik: Yine bir sürü fonksiyon türedi. Öyleyse bunu da genel hâle getirelim.

14.2 İlk Deneme ve İki Engel

`void *arr` ve eleman sayısı alıp `swap`'i `sizeof(arr)` ile çağırmak işe yaramaz. İki engel vardır:

- **Boyut bulunamaz.** `arr`, `void*` türünde olduğu için `sizeof(arr)` sabit bir değerdir. Oysa karakterse 1, `short` ise 2, `int` ise 4 olması gerekir. `void*` ile bu bilgi kaybolur. Çözüm, `swap`'teki gibi, boyutu ayrı bir parametre olarak almaktır: `elem_bytes`.
- **`void*` üzerinde aritmetik yapılamaz.** Bir elemanın adresini `arr + (nelems - 1)` ile bulamayız. C'nin bunun için özel bir olanağı yoktur.

  > **Ders sonrası not:** Hocanın "sabit" dediği değer gcc ile denendi: 20 baytlık `int` dizisi ve 5 baytlık `char` dizisi için `void*` parametre üzerinden `sizeof(arr)` iki durumda da 8 çıktı.
  >
  > **Ek bilgi (derste anlatılmadı):** gcc ile denendi: `void *p` için `p + 4` yazımı `gcc -Wall` altında uyarısız derlendi ve bayt bayt ilerledi, `gcc -Wall -pedantic` ise "pointer of type 'void *' used in arithmetic" uyarısı verdi. Büyük olasılıkla bu gcc'ye özgü bir uzantıdır ve standart C değildir. Bu yüzden derleyicinizde çalışıyor görünse de hocanın anlattığı `char*` yolunu kullanın. `*p` ile dereferans ise gcc'de hata verdi.

14.3 Çözüm: char* ve Eleman Boyutuyla Çarpma

Hangi dizi gelirse gelsin onu karakter dizisine dönüştürürüz, çünkü temel birimimiz karakterdir ve karakter birer birer ilerler. n'inci elemana ulaşmak için indeksi bayt cinsine çevirmek, yani eleman boyutuyla çarpmak gerekir. `int` ise 4 ile, `short` ise 2 ile çarparız, yapı dizisiyse yapının boyutu gelir.

```
void swap_ends(void *arr, size_t nelems, size_t elem_bytes) {
    swap(arr, (char *)arr + (nelems - 1) * elem_bytes, elem_bytes);
}
```

Çağrı örneği beş elemanlı bir `int` dizisidir:

```
int nums[] = {5, 2, 3, 4, 1};
swap_ends(nums, sizeof(nums) / sizeof(nums[0]), sizeof(nums[0]));
```

Adım adım (ara adım eklendi):

- **Verilen:** $n_{elems} = 5$, `elem_bytes` $= 4$.

- **Son elemanın uzaklığı:** $(5-1) \times 4 = 16$ bayt. Python ile hesaplandı. Aynı formül `short` için 8, `char` için 4 bayt verir.
- **Sonuç:** İlk eleman 5 ile son eleman 1 yer değiştirir ve dizi $\{1, 2, 3, 4, 5\}$ olur. gcc ile derlenip çalıştırıldı.

Hoca, aynı yolla bir dizinin yalnız ilk ve son elemanını değil, herhangi iki elemanını da genel olarak değiştirebileceğinizi söyledi. Slaytlarda `short`, karakter ve yapı dizileri için örnekler de var.

15. Dizi Döndürme ve Genetik Algoritma Örneği

Sonraki uygulama dizi döndürme (rotation) problemidir. Hoca bunu bir dizinin içinden bir noktayı alıp parçaları yer değiştirmek olarak anlattı ve genetik algoritmadaki çaprazlamaya benzetti. Biyolojiden hatırladığımız genlerin çaprazlanması gibi, iki gen bilgisinin parçaları takas edilip yeni nesil üretilir.

Hocanın örneği (ara adım eklendi):

- **Verilen:** `0110` ve `1100`.
- **Parçalama:** Her biri ortadan ikiye ayrılır: `01|10` ve `11|00`.
- **Takas:** Birincinin başı ile ikincinin sonu `0100` verir, ikincinin başı ile birincinin sonu `1110` verir.
- **Sonuç:** Yeni nesil `0100` ve `1110` olur. Python ile hesaplandı, hocanın söylediği değerlerle aynı.

Döndürmede ise dizi kendi üzerinde işlenir. Hoca `1, 2, 3, 4, 5, 6` dizisini "üç birim döndür" diye örnekledi ve bunun yukarıdaki fonksiyonlarla nasıl yapılacağını slaytlara bıraktı. Sonuç derste söylenmedi (ara adım eklendi): Dizi $\{4, 5, 6, 1, 2, 3\}$ olur. Python ile hesaplandı. Altı elemanda üç birim döndürmede sola ve sağa döndürme aynı sonucu verir, yön farkı burada önemli değildir.

16. Genel Kabarcık Sıralaması

16.1 Amaç

Kabarcık sıralamasını (bubble sort) `int` için kullanıyoruz, ama karakter ya da yapı dizileri için de gerekebilir. `intBubbleSort` gibi ayrı ayrı yazmak yerine genel bir tane istiyoruz, hatta bunu `bubblesort.h` diye bir kütüphane yapıp her türden veriyi sıralatabiliriz. Hoca, artan ve azalan sıralamanın bizim daha önce yaptığımız örnekle aynı olduğunu söyledi.

İmza değişir: Dizi artık `void*` gelir. Eleman sayısının yanında her elemanın bayt cinsinden boyutu da alınır. Bir de karşılaştırma fonksiyonu alınır. Çünkü bir nesneyi karşılaştırırken kişilerin kilolarını ya da boylarını, bir polinomun derecesini karşılaştırabiliriz. Karşılaştırma ölçütü dışarıdan verilince artan ya da azalan sıralama da bu fonksiyonla seçilir.

16.2 Erken Durma ve Bayrak Değişkeni

Hoca burada bir ayrıntı verdi ve şu diziyi sordu: 2, 5, 4, 6, 7. Bu diziyi sıralamak için tek bir `for` döngüsü yeterli olur mu?

Soru: Bir kez dönmek yeter mi? Öğrenciler önce "olmaz", sonra "4 ile 5'in yeri değişiyor" dedi.
Cevap: Bütün diziyi yine de kontrol etmemiz gerekir.

İzlenen adımlar:

Birinci tur:

- 2 ile 5: sıra bozuk değil.
- 5 ile 4: sıra bozuk, yer değiştir. Dizi 2, 4, 5, 6, 7 olur.
- 5 ile 6: sıra bozuk değil.
- 6 ile 7: sıra bozuk değil.

Birinci turda bir yer değiştirme yapıldığı için bir tur daha gidilir. İkinci tur:

- 2 ile 4, 4 ile 5, 5 ile 6, 6 ile 7: hiçbirinde değişiklik yok.

Python ile hesaplandı: Birinci tur 4 karşılaştırma ve 1 değişiklik, ikinci tur 4 karşılaştırma ve 0 değişiklik, toplam 8 karşılaştırma. Bu, $n^2 = 25$ değerinin altındadır.

Bir tur boyunca, yani dizinin başından sonuna giderken hiç yer değiştirme yapılmadıysa dizi sıralıdır. Bunu anlamak için bir bayrak (flag) değişkeni, `swapped`, kullanılır. Böylece n^2'den daha az karşılaştırmayla algoritma sonlandırılabilir. Değişken `bool` olmak zorunda değildir, yerine `int` de kullanılabilir.

Hoca, slayttaki "hiç değiştirmediysen `return` yap" yerine kendisinin dış döngüyü bayrağa bağlamayı yeğlediğini söyledi. Bayrak her turun başında yanlış (0) yapılır, dizi en az bir kez kontrol edilir.

> **Ders sonrası not:** Hoca ikinci turu anlatırken "5 ile 6'ya baktık, yer değiştirdik" dedi, ama 5 ile 6 sıralı olduğu için değişiklik olmaz. Ayrıca "`swapped` doğru olduysa bu iş bitsin" ifadesi geçti, hemen ardından "tekrar dönsün" dendi. Doğrusu ikincisidir: `swapped` doğru olduysa bir tur daha dönülür, turun sonunda yanlış kaldıysa algoritma biter. Aşağıdaki kod gcc ile böyle çalıştırıldı.

16.3 Elemanların Adresi ve Kod

`swap` zaten genel olduğu için dizinin i'inci ve $(i-1)$'inci elemanlarını onunla değiştirebiliriz. Tek sorun, bu elemanların adresini hesaplamaktır. `void*` üzerinde aritmetik olmadığından dizi `char*`'a çevrilir ve `(char *)arr + (i - 1) * elem_bytes` ile önceki, `(char *)arr + i * elem_bytes` ile mevcut eleman bulunur. Bunlar `void*` değişkenlerde tutulur ve `swap`'e verilir. Karşılaştırma fonksiyonu da `void*` alacak biçimde genel yapılınca iş tamamlanır.

Slaytta yazılı son hâl altyazıdan tam okunmadığı için, kısa nottaki kod hocanın tercih ettiği döngü biçimine uyarlandı.

```c
#include <stdio.h>
#include <string.h>

void swap(void *data1ptr, void *data2ptr, size_t nbytes) {
    char temp[nbytes];
    memcpy(temp, data1ptr, nbytes);
    memcpy(data1ptr, data2ptr, nbytes);
    memcpy(data2ptr, temp, nbytes);
}

void swap_ends(void *arr, size_t nelems, size_t elem_bytes) {
    swap(arr, (char *)arr + (nelems - 1) * elem_bytes, elem_bytes);
}

void bubble_sort(void *arr, size_t n, size_t elem_bytes,
                 int (*cmp)(const void *, const void *)) {
    int swapped;
    size_t i;

    do {
        swapped = 0;
        for (i = 1; i < n; i++) {
            void *prev = (char *)arr + (i - 1) * elem_bytes;
            void *curr = (char *)arr + i * elem_bytes;
            if (cmp(prev, curr) > 0) {
                swap(prev, curr, elem_bytes);
                swapped = 1;
            }
        }
    } while (swapped);
}

int artan(const void *a, const void *b) {
```

```
    return *(const int *)a - *(const int *)b;
}

int azalan(const void *a, const void *b) {
    return *(const int *)b - *(const int *)a;
}

void yazdir(const int *d, size_t n) {
    size_t i;
    for (i = 0; i < n; i++)
        printf("%d ", d[i]);
    printf("\n");
}

int main(void) {
    int nums[] = {5, 2, 3, 4, 1};
    int dizi[] = {2, 5, 4, 6, 7};
    size_t n1 = sizeof(nums) / sizeof(nums[0]);
    size_t n2 = sizeof(dizi) / sizeof(dizi[0]);

    swap_ends(nums, n1, sizeof(nums[0]));
    yazdir(nums, n1);

    bubble_sort(dizi, n2, sizeof(dizi[0]), artan);
    yazdir(dizi, n2);

    bubble_sort(dizi, n2, sizeof(dizi[0]), azalan);
    yazdir(dizi, n2);
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlendi, çıktısı şöyledir:

```
1 2 3 4 5
2 4 5 6 7
7 6 5 4 2
```

Bir yapı dizisi için yalnız karşılaştırma fonksiyonu değişir, `bubble_sort` aynı kalır. Hocanın dediği gibi kişileri kiloya göre sıralamak için:

```
typedef struct {
    int kilo;
} Kisi;

int kiloya_gore(const void *a, const void *b) {
    return ((const Kisi *)a)->kilo - ((const Kisi *)b)->kilo;
}
```

Yukarıdaki yapı ve karşılaştırma fonksiyonu aynı `bubble_sort` ile gcc'de denendi, sıralama doğru çalıştı.

Hoca, derste gösterdiği hızlı sıralamanın (`qsort`) da bu şekilde genel olduğunu söyledi: Ona `int` verirseniz `int`'e göre, karakter verirseniz karaktere göre çalışır. Slaytta geçen `scandir` gibi hazır fonksiyonlara şu an gerek olmadığını söyleyerek geçti.

17. C'de Tür Bilgisi ve void* Üzerine Son Söz

Java ya da C# gibi yüksek seviyeli dillerde tür bilgisini doğrudan elde edebilirsiniz. Gelen değişkenin türünü sorarsınız ("bu bir `int`") ve türe göre işlem yaparsınız. C'de tür bilgisini elde etmek mümkün değildir, yalnız `sizeof` ile boyut bilinir. Genel fonksiyonlarda boyutu alıp `char*` üzerinden çalışmamızın nedeni budur.

Hoca bu kısmı `void` işaretçileri, fonksiyon işaretçileri ve genel fonksiyonların nasıl kullanılacağına dair güzel örnekler olarak ekstra anlattığını, ayrıntılar için Stanford CS107 slaytlarındaki bağlantılara bakılabileceğini ve bu tür şeylerin ileriki sınıflarda gerekeceğini söyledi.

> **Hocanın vurgusu:** Bu konu yeni değil, mevcut konuların bir uygulamasıdır. Genel bir kabarcık sıralaması yazdırılmaz, ama işaretçilerin birbirine dönüştürülmesi ve `sizeof` ile birlikte kullanılması üzerine ekran çıktısı soruları sorulabilir. Örneğin `void*` alan ve boyut üzerinden işlem yapan bir fonksiyona veri gönderilip sonucun ne olduğu sorulabilir. Bunlara bakmakta fayda var.

Dersin Toparlaması

Bu derste tek akışlı programlardan çok akışlı programlara geçtik. Tek işlemcide zaman paylaşımıyla sanal paralellik, birden çok işlemcide gerçek paralellik elde edildiğini, süreçlerin bellekleri ayrı olduğu için bir süreç içinde iş parçacıkları kullanmanın daha rahat olduğunu gördük. `pthread_create`, `pthread_self`, `pthread_equal`, `pthread_exit` ve `pthread_join` ile iş parçacığı oluşturmayı, tanımayı, değer döndürmeyi ve beklemeyi öğrendik. Çalışma sırasının öngörülemeyeceği bütün örneklerin ortak sonucuydu.

İkinci yarıda aynı `void*` fikri, tür bağımsız fonksiyonlara taşındı. C çalışma anında tür bilgisi vermediği için verinin kaç bayt olduğu ayrıca alınır, kopyalama `memcpy` ile, adres hesabı da `char*` üzerinden eleman boyutuyla çarpılarak yapılır. Karşılaştırma için fonksiyon işaretçisi eklenince `swap`, `swap_ends` ve kabarcık sıralaması her türde çalışır. Hoca, iş parçacıklarının ayrıntılarının İşletim Sistemleri ve Bilgisayar Organizasyonu derslerinde görüleceğini söyleyerek dönemi bitirdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.